

Fundamentos de Lógica Computacional e Introdução à Lógica Proposicional

Lógica Computacional · 3º Semestre | Engenharia de Software

Prof. Me. Keny Lucas da Silva Goes



CAMPUS XXIII
PARAUAPEBAS

OBJETIVOS DA AULA

01

Papel da lógica na computação

Compreender por que a lógica é fundamental para a Engenharia de Software.

02

Proposições

Identificar sentenças que podem ser classificadas como verdadeiras ou falsas.

03

Conectivos lógicos

Conhecer os principais operadores: $\neg$, $\wedge$, $\vee$, $\rightarrow$ e $\leftrightarrow$.

04

Linguagem formal

Representar sentenças da língua natural em notação lógica.

05

Tabelas-verdade

Iniciar a construção e interpretação de tabelas-verdade.

O que é Lógica Computacional?

Definição

Lógica Computacional estuda **estruturas formais** utilizadas para representar raciocínios, validar argumentos, modelar condições e apoiar soluções em computação.

Para que serve?

- Formalizar regras e condições em sistemas
- Verificar a correção de algoritmos
- Modelar o comportamento de software
- Dar base matemática à Engenharia de Software

Por que estudar lógica?



Organiza o pensamento

Permite estruturar ideias de forma clara e sem contradições.



Constrói argumentos válidos

Garante que as conclusões decorram corretamente das premissas.



Resolve problemas

Fornece um método sistemático para decompor e solucionar desafios.



Evita contradições

Identifica inconsistências em regras, sistemas e especificações.

Lógica e Computação

A lógica está presente em praticamente todas as áreas da computação:



Algoritmos



Testes de Software



Bancos de Dados



Inteligência Artificial



Circuitos Digitais



Validação de Requisitos

Lógica no dia a dia do programador

Se o usuário digitou a senha correta **e** está ativo, **então** o acesso é permitido.

Em linguagem natural

Uma regra de negócio simples, expressa em português, que define quando o acesso deve ser liberado.

Em lógica formal

Sendo **P** = "senha correta" e **Q** = "usuário ativo":

$(P \wedge Q) \rightarrow$ Acesso permitido

A lógica fornece a notação precisa para representar essa condição sem ambiguidade.

O que é lógica?

Definição

Lógica é o estudo da **natureza do raciocínio** e do conhecimento. Ela nos permite formalizar e justificar argumentos de forma rigorosa e precisa.

O que a lógica faz?

- Determina se um raciocínio é **válido** ou **inválido**
- Distingue o que é verdadeiro do que é falso
- Fornece regras para derivar conclusões corretas
- Elimina a ambiguidade da linguagem natural

Lógica clássica bivalente

Na lógica clássica, toda sentença assume **exatamente um** de dois valores possíveis:

VERDADEIRO (V / 1)

A sentença descreve um fato que corresponde à realidade.

Exemplo: *"Belém é a capital do Pará."*

FALSO (F / 0)

A sentença descreve um fato que não corresponde à realidade.

Exemplo: *"O Sol gira em torno da Terra."*

Não existe um terceiro valor: a sentença é V **ou** F — nunca os dois ao mesmo tempo.

Exemplo de raciocínio lógico

Premissa 1

Se está chovendo, então a rua está molhada.

Premissa 2

Está chovendo.

Conclusão

Logo, a rua está molhada.

A conclusão é uma consequência **necessária** das premissas. Se as premissas forem verdadeiras, a conclusão **não pode** ser falsa. Isso é o que chamamos de argumento **válido**.

O que é Lógica Proposicional?

É o ramo mais fundamental da lógica, no qual fatos do mundo real são representados por sentenças declarativas chamadas **proposições**.



Forma mais simples

Trabalha com proposições inteiras, sem analisar sua estrutura interna.



Base de tudo

É o ponto de partida para lógica de predicados, álgebra booleana e muito mais.



Combina proposições

Usa conectivos para formar expressões compostas mais complexas.

Do mundo real para a proposição lógica

Cada sentença do cotidiano pode ser traduzida para um símbolo lógico:

Sentença em português	Símbolo	Notação lógica
Hoje está chovendo.	P	P
A rua está molhada.	Q	Q
Se está chovendo, então a rua está molhada.	$P \rightarrow Q$	$P \rightarrow Q$

O que é uma proposição?

Proposição é uma sentença declarativa que pode ser classificada, de forma não ambígua, como **verdadeira** ou **falsa** — nunca os dois ao mesmo tempo.

✓ É proposição

Sentença declarativa com valor de verdade definido.

❌ Não é proposição

Perguntas, ordens, exclamações ou sentenças paradoxais.

Exemplos de proposições

Sentença	Valor	Por quê?
$1 + 1 = 2$	✓ Verdadeiro	Fato matemático
$0 > 1$	✗ Falso	0 não é maior que 1
Belém é uma cidade do Pará.	✓ Verdadeiro	Fato geográfico
Todo número par é divisível por 2.	✓ Verdadeiro	Definição matemática

Sentenças que *não* são proposições

Nem toda sentença da língua portuguesa é uma proposição lógica. Observe os casos abaixo:

? Perguntas

"Qual é o seu nome?" — não pode ser classificada como verdadeira ou falsa.

! Ordens / Imperativos

"Preste atenção!" — expressa um comando, não um fato verificável.

↺↻ Paradoxos

"Esta frase é falsa." — se for verdadeira, é falsa; se for falsa, é verdadeira. Não tem valor definido.

É proposição ou não?

Classifique cada sentença abaixo como **Proposição (P)** ou **Não é proposição (N)**:

1

Boa sorte!

2

Cecília é escritora.

3

Quantos alunos estão na sala?

4

$2 + 3 = 5$

5

Não faça isto!

Respostas

Sentença	Classificação	Justificativa
Boa sorte!	✗ Não é proposição	Exclamação, não tem valor de verdade
Cecília é escritora.	✓ Proposição	Sentença declarativa verificável
Quantos alunos estão na sala?	✗ Não é proposição	É uma pergunta
$2 + 3 = 5$	✓ Proposição	Fato matemático – verdadeiro
Não faça isto!	✗ Não é proposição	Ordem imperativa

Símbolos proposicionais

Convenção

Proposições são representadas por **letras maiúsculas**: P, Q, R, S, T, ...

Cada letra substitui uma sentença completa, funcionando como uma variável lógica.

Exemplos práticos

- **P** = "O sistema está online."
- **Q** = "O usuário está autenticado."
- **R** = "O banco de dados está acessível."
- **S** = "O backup foi concluído."

Com esses símbolos, podemos construir expressões como: **$P \wedge Q \rightarrow R$**

Os 5 conectivos fundamentais

Símbolo	Nome	Lê-se	Exemplo
¬	Negação	"não"	¬P
∧	Conjunção	"e"	P ∧ Q
∨	Disjunção	"ou"	P ∨ Q
→	Implicação	"se ... então"	P → Q
↔	Bicondicional	"se e somente se"	P ↔ Q

Negação — $\neg$

Como funciona

A negação é um conectivo **unário**: atua sobre uma única proposição e **inverte** seu valor lógico.

$V \rightarrow F \cdot F \rightarrow V$

Exemplo computacional

P = "O sistema está ativo." (V)

$\neg P$ = "O sistema *não* está ativo." (F)

Na programação

```
if (!sistemaAtivo) { ... }
```

O operador **!** é a negação lógica.

Conjunção — $\wedge$

Regra

$P \wedge Q$ é verdadeiro somente quando P e Q são ambas verdadeiras. Basta uma ser falsa para o resultado ser falso.

Exemplo

P = "O usuário está autenticado."

Q = "O cadastro está ativo."

$P \wedge Q$ = "O usuário está autenticado e o cadastro está ativo."

Em código: `if (autenticado && cadastroAtivo)`

Disjunção — $\vee$

Regra

$P \vee Q$ é verdadeiro quando pelo menos uma das proposições é verdadeira. Só é falso quando **ambas** são falsas.

Exemplo

P = "O usuário pode acessar por senha."

Q = "O usuário pode acessar por biometria."

$P \vee Q$ = "O usuário acessa por senha *ou* por biometria."

Em código: `if (senha || biometria)`

Implicação — $\rightarrow$

Estrutura

$P \rightarrow Q$: se P (antecedente) é verdadeiro, então Q (consequente) deve ser verdadeiro.

⚠ Caso crítico: $V \rightarrow F = F$

Todos os outros casos resultam em verdadeiro.

Exemplo

P = "O pagamento foi confirmado."

Q = "O pedido será liberado."

$P \rightarrow Q$ = "Se o pagamento foi confirmado, *então* o pedido será liberado."

Se o pagamento foi feito (V) mas o pedido não foi liberado (F), a regra foi **violada**.

Bicondicional —

Regra

$P \leftrightarrow Q$ é verdadeiro quando P e Q têm o mesmo valor lógico: ambas verdadeiras ou ambas falsas.

$V \leftrightarrow V = V \cdot F \leftrightarrow F = V$

$V \leftrightarrow F = F \cdot F \leftrightarrow V = F$

Exemplo

P = "O aluno cumpriu os critérios."

Q = "O aluno é aprovado."

$P \leftrightarrow Q$ = "O aluno é aprovado *se e somente se* cumpriu os critérios definidos."

Equivalência exata: aprovação ocorre sempre que — e apenas quando — os critérios são cumpridos.

Simbolizando sentenças

Defina símbolos para as proposições e reescreva cada sentença em **notação lógica**:

1

O sistema está ativo e o usuário está autenticado.

2

Se a senha estiver correta, *então* o acesso será permitido.

3

O aluno entrega a atividade *ou* realiza a prova.

Respostas

Sentença	Fórmula	Definições
Sistema ativo e usuário autenticado	$P \wedge Q$	P = sistema ativo; Q = usuário autenticado
Se senha correta, então acesso permitido	$P \rightarrow Q$	P = senha correta; Q = acesso permitido
Entrega atividade ou realiza prova	$P \vee Q$	P = entrega atividade; Q = realiza prova

A linguagem da lógica proposicional

Assim como linguagens de programação têm uma sintaxe própria, a lógica proposicional possui uma **linguagem formal** rigorosa:



Símbolos verdade

True (V, 1) e False (F, 0)



Conectivos

$\neg$, $\wedge$, $\vee$, $\rightarrow$, 



Símbolos proposicionais

Letras maiúsculas: P, Q, R, S, ...



Pontuação

Parênteses () para indicar precedência e agrupamento

Alfabeto da lógica proposicional

Símbolos básicos

- Verdade: **True**, **False** (V, F, 1, 0)
- Proposicionais: **P**, **Q**, **R**, **S**, **T**, ...

Conectivos

- $\neg$ negação
- $\wedge$ conjunção
- $\vee$ disjunção
- $\rightarrow$ implicação
-  bicondicional

Pontuação

- Parênteses: **()**
-

Observação importante

Apenas combinações **válidas** desses elementos formam fórmulas lógicas aceitas. Qualquer outro símbolo fora deste alfabeto **não pertence** à linguagem.

Fórmulas proposicionais

Uma **fórmula proposicional** (ou fbf — fórmula bem formada) é qualquer construção **sintaticamente válida** usando os símbolos do alfabeto:

P

Um símbolo proposicional simples já é uma fórmula.

$\neg P$

Negação de uma fórmula é uma fórmula.

$P \wedge Q$

Conjunção de duas fórmulas é uma fórmula.

$(P \vee Q) \rightarrow R$

Fórmulas mais complexas combinam conectivos e parênteses.

Fórmulas válidas e inválidas

✓ Fórmulas válidas

- $P \vee Q$
- $\neg P$
- $(P \wedge Q) \rightarrow R$
- $\neg(P \leftrightarrow Q)$

✗ Fórmulas inválidas

- PQR – sem conectivos entre os símbolos
- $\rightarrow PQ$ – conectivo no início, sem antecedente
- $P \wedge \vee Q$ – dois conectivos consecutivos
- $P(\wedge Q)$ – parêntese mal posicionado

Nem toda sequência de símbolos forma uma fórmula – a sintaxe deve ser respeitada.

Regras básicas de construção

1 Base

Todo símbolo proposicional ($P, Q, R, \dots$) e todo símbolo verdade (True, False) é uma fórmula.

2 Negação

Se φ é fórmula, então $\neg\varphi$ também é fórmula.

3 Conectivos binários

Se φ e ψ são fórmulas, então $(\varphi \wedge \psi)$, $(\varphi \vee \psi)$, $(\varphi \rightarrow \psi)$ e $(\varphi \leftrightarrow \psi)$ também são fórmulas.

4 Exclusividade

Nada mais é fórmula. Toda fórmula válida é obtida pela aplicação finita dessas regras.

Precedência dos conectivos

Assim como na aritmética ($\times$ antes de $+$), os conectivos têm **ordem de avaliação**:

1



Bicondicional

Menor precedência – avaliado por último

2

→ Implicação

Precedência intermediária

3

∨ Disjunção

Precedência média-alta

4

∧ Conjunção

Precedência alta

5

¬ Negação

Maior precedência – avaliada primeiro

Exemplo de precedência

Veja como a precedência altera a leitura de uma fórmula:

Fórmula	Leitura com precedência	Com parênteses explícitos
$\neg P \wedge Q$	$\neg$ age só em P; depois aplica $\wedge$	$(\neg P) \wedge Q$
$P \vee Q \rightarrow R$	$\vee$ antes de $\rightarrow$	$(P \vee Q) \rightarrow R$
$\neg P \vee Q \wedge R$	$\neg$ primeiro, depois $\wedge$, depois $\vee$	$(\neg P) \vee (Q \wedge R)$

Introdução às Tabelas-Verdade

Quantas linhas tem uma tabela-verdade?

Regra geral

Para **N** proposições distintas, a tabela possui:

$$2^N \text{ linhas}$$

Cada linha representa uma combinação possível de valores V/F.

Exemplos

- **1 proposição** $\rightarrow 2^1 = 2$ linhas
- **2 proposições** $\rightarrow 2^2 = 4$ linhas
- **3 proposições** $\rightarrow 2^3 = 8$ linhas
- **4 proposições** $\rightarrow 2^4 = 16$ linhas

Tabela-verdade da negação — $\neg P$

P	$\neg P$
V	F
F	V

Interpretação

A negação **sempre inverte** o valor lógico da proposição:

- Se P é **verdadeiro**, $\neg P$ é **falso**
- Se P é **falso**, $\neg P$ é **verdadeiro**

Com 1 proposição, a tabela tem **$2^1 = 2$** linhas.

Tabela-verdade da conjunção — $P \wedge Q$

P	Q	$P \wedge Q$
V	V	V
V	F	F
F	V	F
F	F	F

Interpretação

$P \wedge Q$ é verdadeiro apenas na primeira linha — quando **ambas** P e Q são verdadeiras.

Qualquer falsidade em P ou Q torna a conjunção falsa.

Com 2 proposições: **$2^2 = 4$** linhas.

Tabela-verdade da disjunção — $P \vee Q$

P	Q	$P \vee Q$
V	V	V
V	F	V
F	V	V
F	F	F

Interpretação

$P \vee Q$ é falso somente na última linha — quando **ambas** P e Q são falsas.

Em todos os outros casos, pelo menos uma é verdadeira e o resultado é verdadeiro.

Com 2 proposições: $2^2 = 4$ linhas.

Tabela-verdade da implicação — $P \rightarrow Q$

P	Q	$P \rightarrow Q$
V	V	V
V	F	 F
F	V	V
F	F	V

Caso crítico

$V \rightarrow F = F$: a promessa foi feita (P é verdadeiro) mas não foi cumprida (Q é falso). A implicação foi violada.

Demais casos

Quando P é falso, a implicação é **trivialmente verdadeira** — não houve promessa a ser violada.

Consolidando o aprendizado

1

Identificar proposições

Classifique 5 sentenças escolhidas pelo professor como proposição ou não proposição, justificando cada resposta.

2

Simbolizar sentenças

Escolha 3 sentenças do cotidiano computacional, defina os símbolos e escreva as fórmulas lógicas correspondentes.

3

Construir tabelas-verdade

Monte, individualmente, as tabelas-verdade de: $\neg P$, $P \wedge Q$ e $P \vee Q$.